

AroundMe

Costi – Iulian Asanache | Nam Khanh LE | Joseph Ayoub

AroundMe is an Android application that allows users to discover events near their GPS location. It fetches real data from the Ticketmaster Discovery API, displays events as markers on a Google Map, and provides a list view with category filtering. Users can save events to a favourites list and receive a notification before a saved event starts. The app is written in Java, targeting SDK 36, minimum SDK 24.

1. Requirements Mapping

Req.	Implementation
R1	Four Activities (Main, List, Detail, Favorites) connected by explicit Intents. Directions button uses implicit geo: URI Intent to open Google
R2	EventListActivity switches from LinearLayoutManager (portrait) to GridLayoutManager 2 columns (landscape). ViewModel preserves stat
R3	SupportMapFragment embedded in MainActivity. Data exchanged via MapViewModel (shared ViewModel pattern).
R4	menu_main.xml: category filter + open list. menu_event_list.xml: category filter + favorites. All items trigger real actions.
R5	RecyclerView with custom EventAdapter, EventViewHolder, and item_event.xml in EventListActivity and FavoritesActivity.
R6	Theme in themes.xml, styles in styles.xml, all colors in colors.xml. No hard-coded hex values in any layout file.
R7	GPS via FusedLocationProviderClient. ACCESS_FINE_LOCATION requested at runtime. Coordinates used for map and API call.
R8	Foreground Service (NotificationService) monitors saved events every 60 seconds. Fires alert notification 30 minutes before start.

2. Architecture: Component Diagram

The application follows the MVVM pattern. Activities are the View layer. ViewModels hold UI state. EventRepository is the single data source, connecting the Retrofit API client and the Room database.

Layer	Class	Responsibility
View	MainActivity	Map display, GPS permission, marker tap, action bar menu
View	EventListActivity	RecyclerView list, category filter, orientation handling
View	EventDetailActivity	Event details, save/remove favorite, implicit Intent directions
View	FavoritesActivity	List of saved events, remove favorites
ViewModel	MapViewModel	Holds events list, selected event, loading and error state
ViewModel	EventListViewModel	Holds filtered event list, applies category filter
ViewModel	FavoritesViewModel	Exposes LiveData from Room database to FavoritesActivity
Repository	EventRepository	Single source of truth: API calls and database operations
Remote	TicketmasterApi	Retrofit interface: GET events.json with query parameters
Remote	ApiClient	Retrofit singleton with base URL and Gson converter
Remote	EventMapper	Converts raw API DTO objects into Event model objects
Local	AppDatabase	Room singleton, one table storing favorite Event objects
Local	EventDao	Insert, delete, getAllFavorites (LiveData), getFavoriteById
Model	Event	Room @Entity and data model used across all layers
Service	NotificationService	Foreground Service: monitors favorites, fires alert notifications
Util	NotificationHelper	Creates notification channels required on Android 8 and above

Data Flow Summary

MainActivity gets GPS coordinates and calls EventRepository.fetchNearbyEvents(). Retrofit calls the Ticketmaster API. EventMapper converts the response to Event objects. MapViewModel receives the list via LiveData and MainActivity places markers. When the user saves a favourite, EventRepository inserts it into Room and starts NotificationService.

3. Data Model

The central class is `Event.java`, annotated as a Room `@Entity`. The same object is used throughout the app: returned by the API mapper, displayed in the `RecyclerView`, stored in Room, and passed between Activities via Intent extras.

Field	Type	Source / Notes
id	String (@PrimaryKey)	Ticketmaster event ID, unique, never null
name	String	Event name from API
date	String	Local date
time	String	Local time
venueName	String	Venue name from API embedded venues array
venueAddress	String	Street address of the venue
latitude / longitude	double	Venue GPS coordinates from API
category	String	Segment name: Music, Sports, Arts, etc.
imageUrl	String	First image URL from API images array
distanceKm	double	Computed at runtime, not stored in Room

Room Database

`AppDatabase` (version 1) has one table: the events table, used only for favorites. `EventDao` provides: `@Insert` (replace on conflict), `@Delete`, `@Query` `getAllFavorites()` returning `LiveData`, `@Query` `getAllFavoritesNow()` returning a plain List for background threads, and `@Query` `getFavoriteById()` to check if an event is already saved before showing the star as filled.

API DTOs

Three DTO classes mirror the Ticketmaster JSON structure: `TicketmasterResponse` (top level with `_embedded`), `TmEmbedded` (contains the events array), and `TmEvent` (one event with nested venues, images, classifications, and dates). Gson parses these automatically. `EventMapper.fromTmEventList()` converts each `TmEvent` into a clean `Event` object, extracting the first venue, the first image, the segment name as category, and the local date and time.

4. R8: Foreground Service Justification

Why not BroadcastReceiver

A `BroadcastReceiver` responds to one-shot system events and is limited to about 10 seconds of execution. It cannot run a periodic monitoring loop, so it cannot check event times repeatedly.

Why not ContentProvider

A `ContentProvider` is designed to share data between applications. [AroundMe](#) does not expose data to other apps, so a `ContentProvider` adds no value here.

Why a foreground Service

A foreground Service runs independently of the UI lifecycle. When the user closes the app, the service keeps running because it holds a persistent notification. A background Service would be killed by Android battery optimization within minutes on modern devices, making notification scheduling unreliable.

How NotificationService works

- Started by EventListActivity when user saves a favorite via ContextCompat.startForegroundService().
 - onStartCommand() calls startForeground() immediately with a persistent low-priority notification.
 - A Handler posts checkFavoriteEvents() on a background thread every 60 seconds.
 - checkFavoriteEvents() calls EventDao.getAllFavoritesNow() and calls minutesUntilEvent() for each.
-
- minutesUntilEvent() parses the event date + time string and computes minutes remaining.
 - If an event is 0 to 30 minutes away, sendEventAlert() fires a high-priority notification.
 - If no upcoming events remain, stopSelf() shuts the service down cleanly.
 - onDestroy() removes the Handler callbacks to avoid memory leaks.

foregroundServiceType

The service is declared with android:foregroundServiceType="dataSync" in AndroidManifest.xml because it reads from the local database. This is required by Android 14 and above.

5. Permissions

Permission	Why it is needed
ACCESS_FINE_LOCATION	Precise GPS via FusedLocationProviderClient. Requested at runtime in MainActivity.
ACCESS_COARSE_LOCATION	Declared alongside fine location as required by Android when requesting location.
INTERNET	All network calls: Ticketmaster API and Google Maps tile loading.
FOREGROUND_SERVICE	Required to call startForeground() without a SecurityException.
FOREGROUND_SERVICE_DATA_SYNC	Required on Android 14+ for a foreground service of type dataSync.
POST_NOTIFICATIONS	Required on Android 13+ to show any notification. Requested at runtime in MainActivity.

6. Application Screenshots

The screenshots below show the application running on a real Samsung Galaxy S23 Ultra (Figures 1 and 3) and on the Android emulator Pixel 9 Pro XL .



